Encontro 8: Reatividade Avançada e Gerenciamento de Estado com Vuex

Sumário

Encontro 8: Reatividade Avançada e Gerenciamento de Estado com Vuex 1

1. Introdução 2
   1.1. Gerenciamento de estados em aplicações Vue 2
2. O que é Vuex? 3
3. Os Conceitos Fundamentais do Vuex 4
4. Configurando um Store Vuex 5
5. Exercício Prático Integrado: Carrinho de Compras Global 6

1. Introdução

O gerenciamento de estados no Vue.js (geralmente usando **Pinia** ou **Vuex**) resolve, principalmente, problemas de comunicação complexa e compartilhamento de dados entre componentes em aplicações de médio a grande porte.

As situações práticas mais comuns incluem:

- Compartilhamento de dados entre componentes não relacionados: Em uma aplicação grande, passar dados entre componentes que não são pai e filho diretos (comunicação via props e events) torna-se complexo e ineficiente, criando a necessidade de "prop drilling" (passar props por muitos níveis). O gerenciamento de estado centralizado resolve isso, permitindo que qualquer componente acesse ou modifique o estado global.
- Gestão de estado global da aplicação: Informações que precisam estar disponíveis em múltiplos locais da aplicação, como dados do usuário autenticado, preferências de tema (claro/escuro), ou configurações de idioma, são idealmente armazenadas em um store centralizado.
- Carrinhos de compras: Em e-commerces, o conteúdo do carrinho de compras precisa ser acessado e atualizado a partir de diferentes visualizações (listagem de produtos, página do produto, ícone do carrinho no cabeçalho, página de checkout). O gerenciamento de estado global simplifica essa coordenação.
- Dados de formulários multi-etapas: Quando um formulário complexo é dividido em vários componentes ou páginas, o estado centralizado permite reter os dados preenchidos pelo usuário à medida que ele avança pelas etapas.
- Resultados de API e cache de dados: Dados complexos obtidos de APIs, que são usados por múltiplos componentes, podem ser armazenados em um store para evitar requisições repetitivas ao servidor, melhorando a performance e a experiência do usuário.
- Previsibilidade nas mutações de estado: O uso de bibliotecas como **Vuex** ou **Pinia** impõe regras e padrões para a alteração do estado (como o padrão **Flux**), o que torna mais fácil depurar e entender de onde vêm as mudanças nos dados, especialmente com o uso das ferramentas de desenvolvimento (Vue DevTools).

O gerenciamento de estado global se torna essencial quando a complexidade da comunicação entre componentes começa a ofuscar a lógica de negócios da aplicação, proporcionando uma fonte de verdade única, reativa e previsível para todos os dados compartilhados.

1.1. Gerenciamento de estados em aplicações Vue

Em aplicações Vue, cada componente tem seu próprio estado local, gerenciado pela opção data. Isso funciona bem para componentes isolados, mas à medida que a aplicação cresce, surgem desafios:

- Compartilhamento de Estado: Como múltiplos componentes, que não são diretamente pai e filho, podem acessar e modificar o mesmo dado (ex: informações do usuário logado)?
- Prop Drilling: Para passar um dado de um componente avô para um componente neto, somos forçados a passar a prop por todos os componentes intermediários, mesmo que eles não a utilizem. Isso torna a manutenção complexa e frágil.

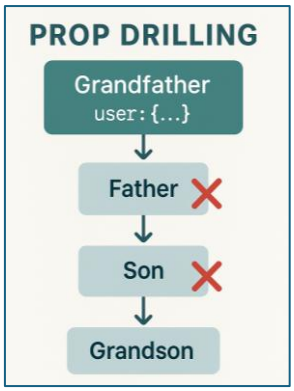


Figura 1: Prop do avô usada pelo neto

A figura 1 ilustra o prop drilling: uma propriedade precisa ser passada por múltiplos níveis de componentes para chegar ao seu destino. Para resolver, usamos um padrão de gerenciamento de estado centralizado. No ecossistema Vue, a ferramenta clássica para isso é o **Vuex**.

Nas aplicações **Vue 3**, a biblioteca oficialmente recomendada agora é o **Pinia**, que oferece uma API mais simples e modular. Ainda é necessário entender o **Vuex**, pois ele ainda é usado em milhares de projetos e seus conceitos fundamentais formaram a base para o **Pinia**.

2. O que é Vuex?

Vuex é uma biblioteca que implementa um `store` (armazém) centralizado para todos os componentes de uma aplicação. Ele garante que o estado só possa ser modificado de forma previsível, seguindo um conjunto de regras. Esse estado centralizado se torna a única fonte da verdade (Single Source of Truth).

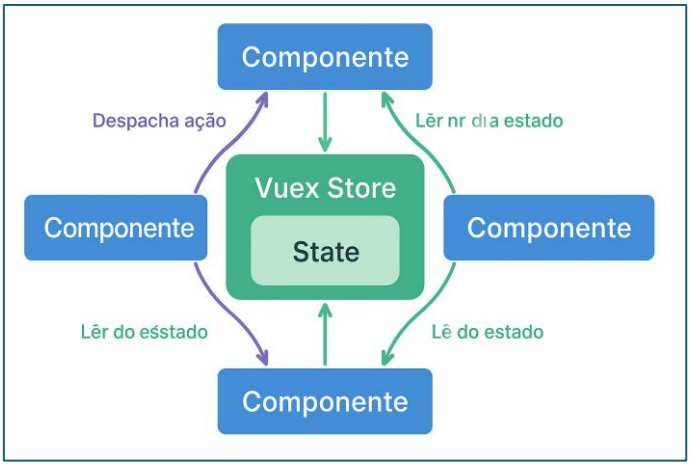


Figura 2: Passagem de props com o Vuex

A figura 2 mostra que, com o Vuex, os componentes não se comunicam mais diretamente. Eles leem o estado do Store central e disparam ações para modificá-lo.

3. Os Conceitos Fundamentais do Vuex

O Vuex opera com base em quatro conceitos principais, que garantem um fluxo de dados unidirecional e previsível.

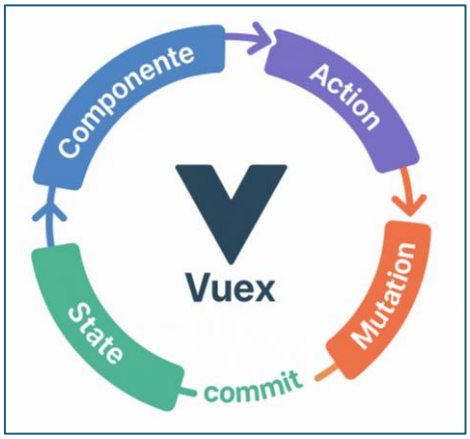


Figura 3: O fluxo de dados em Vuex

A figura 3 mostra o fluxo de dados em Vuex: um componente dispara uma Ação, que comita uma Mutação, que altera o Estado, e a mudança é refletida de volta no componente.

a) `State`: O objeto que contém os dados da sua aplicação. É a única fonte da verdade. Qualquer componente pode ler diretamente deste estado.

b) `Getters`: São como as computed properties para o store. Permitem derivar um estado com base no estado do store (ex: filtrar uma lista de tarefas, contar itens).

c) `Mutations`: A única forma de alterar o estado no Vuex. São funções síncronas que recebem o estado como primeiro argumento. Em vez de modificar o estado diretamente, você "comita" (confirma) uma mutação.

d) `Actions`: Semelhantes às mutações, mas com duas diferenças cruciais:

- Em vez de modificar o estado, elas comitam mutações.
- Podem conter operações assíncronas (como chamadas de API).

4. Configurando um Store Vuex

Primeiro, instale o Vuex:

```
npm install vuex@next --save
```

Depois, crie o store, geralmente em um arquivo `src/store/index.js`:

```
import { createStore } from 'vuex'
// Crie uma nova instância de store.
const store = createStore({
  state() {
    return {
      contador: 0
    }
  },
  mutations: {
    incrementar(state) {
      state.contador++
    },
    adicionar(state, payload) {
      state.contador += payload.valor;
    }
  },
  actions: {
    incrementarAssincrono(context) {
      setTimeout(() => {
        context.commit('incrementar')
      }, 1000)
    }
  },
  getters: {
    contadorDobrado(state) {
      return state.contador * 2;
    }
  }
})
export default store;
```

Finalmente, no seu `main.js`, você precisa dizer à sua aplicação Vue para usar o store:

```
import { createApp } from 'vue'
import App from './App.vue'
import store from './store' // Importe o store

const app = createApp(App)

app.use(store) // Use o store
app.mount('#app')
```

5. Exercício Prático Integrado: Carrinho de Compras Global

Objetivo: Criar uma miniloja com um componente de produtos e um componente de cabeçalho que exibe o número de itens no carrinho. O estado do carrinho será gerenciado pelo Vuex.

Passo 1: Configurar o Store (`src/store/index.js`)

```
import { createStore } from 'vuex';
const store = createStore({
  state() {
    return {
      produtosNoCarrinho: []
    };
  },
  mutations: {
    ADICIONAR_PRODUTO(state, produto) {
      state.produtosNoCarrinho.push(produto);
    }
  },
  actions: {
    adicionarProdutoAoCarrinho(context, produto) {
      // Aqui poderíamos ter lógica assíncrona, validação, etc.
      context.commit('ADICIONAR_PRODUTO', produto);
    }
  },
  getters: {
    totalItensNoCarrinho(state) {
      return state.produtosNoCarrinho.length;
    }
  }
});
export default store;
```

Passo 2: Criar os Componentes

`src/components/HeaderLoja.vue` (Lê o estado com um getter)

```
<template>
  <header>
    <h1>Minha Loja</h1>
    <div class="carrinho-info">Carrinho: {{ totalItens }} itens</div>
  </header>
</template>

<script>
export default {
  computed: {
    totalItens() {
      // Acessa o getter do store
      return this.$store.getters.totalItensNoCarrinho;
    }
  }
}
</script>
```

```
<style scoped>
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #2c3e50; /* Fundo escuro */
  color: white; /* Texto branco */
  padding: 15px 20px;
}

.carrinho-info {
  color: white; /* Garante texto branco no carrinho */
}
</style>
```

`src/components/ListaProdutos.vue` (Dispara uma ação)

```
<template>
  <div class="lista-produtos">
    <h2>Produtos</h2>
    <ul>
      <li v-for="produto in produtos" :key="produto.id">
        {{ produto.nome }} - R$ {{ produto.preco.toFixed(2) }}
        <button @click="adicionarAoCarrinho(produto)">
          Adicionar ao Carrinho
        </button>
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      produtos: [
        { id: 1, nome: 'Produto A', preco: 49.90 },
        { id: 2, nome: 'Produto B', preco: 29.90 },
        { id: 3, nome: 'Produto C', preco: 89.90 }
      ]
    };
  },
  methods: {
    adicionarAoCarrinho(produto) {
      // Dispara a ação do store
      this.$store.dispatch('adicionarProdutoAoCarrinho', produto);
    }
  }
}
</script>
```

```
<style scoped>
.lista-produtos {
  max-width: 600px;
  margin: 0 auto;
  padding: 20px;
}

ul {
  list-style: none;
  padding: 0;
}

li {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 15px;
  padding: 20px;
  background: white; /* Fundo branco */
  border: 1px solid #ddd; /* Borda cinza clara */
  border-radius: 8px; /* Bordas arredondadas */
  box-shadow: 0 2px 4px rgba(0,0,0,0.1); /* Sombra suave */
}

button {
  margin-left: 15px;
  padding: 10px 20px;
  background: #42b983; /* Verde Vue.js */
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

button:hover {
  background: #35a372;
}
</style>
```

Passo 3: Montar a Aplicação (`App.vue`)

```
<template>
  <HeaderLoja />
  <main>
    <ListaProdutos />
  </main>
</template>

<script>
import HeaderLoja from './components/HeaderLoja.vue';
import ListaProdutos from './components/ListaProdutos.vue';

export default {
  components: { HeaderLoja, ListaProdutos }
}
</script>

<style scoped>
body {
  margin: 0;
  background: #f5f5f5; /* Fundo cinza claro para a página */
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-
serif;
}

main {
  padding: 0; /* Remover padding para o header ficar full-width */
}
</style>
```

Resultado Final

Ao clicar no botão "Adicionar ao Carrinho" do componente `ListaProdutos`, a ação `adicionarProdutoAoCarrinho` é disparada. Ela comita a mutação `ADICIONAR_PRODUTO`, que atualiza o estado `produtosNoCarrinho`. O `HeaderLoja`, que está observando o getter `totalItensNoCarrinho`, é automaticamente atualizado para refletir a nova contagem de itens, tudo isso sem comunicação direta entre os componentes.



Figura 4: Resultado do exercício

A figura 4 ilustra a aplicação final, onde clicar em "Adicionar ao Carrinho" em um componente atualiza o contador em outro componente através do store Vuex.